

TETRIS++

By *Velvetal*

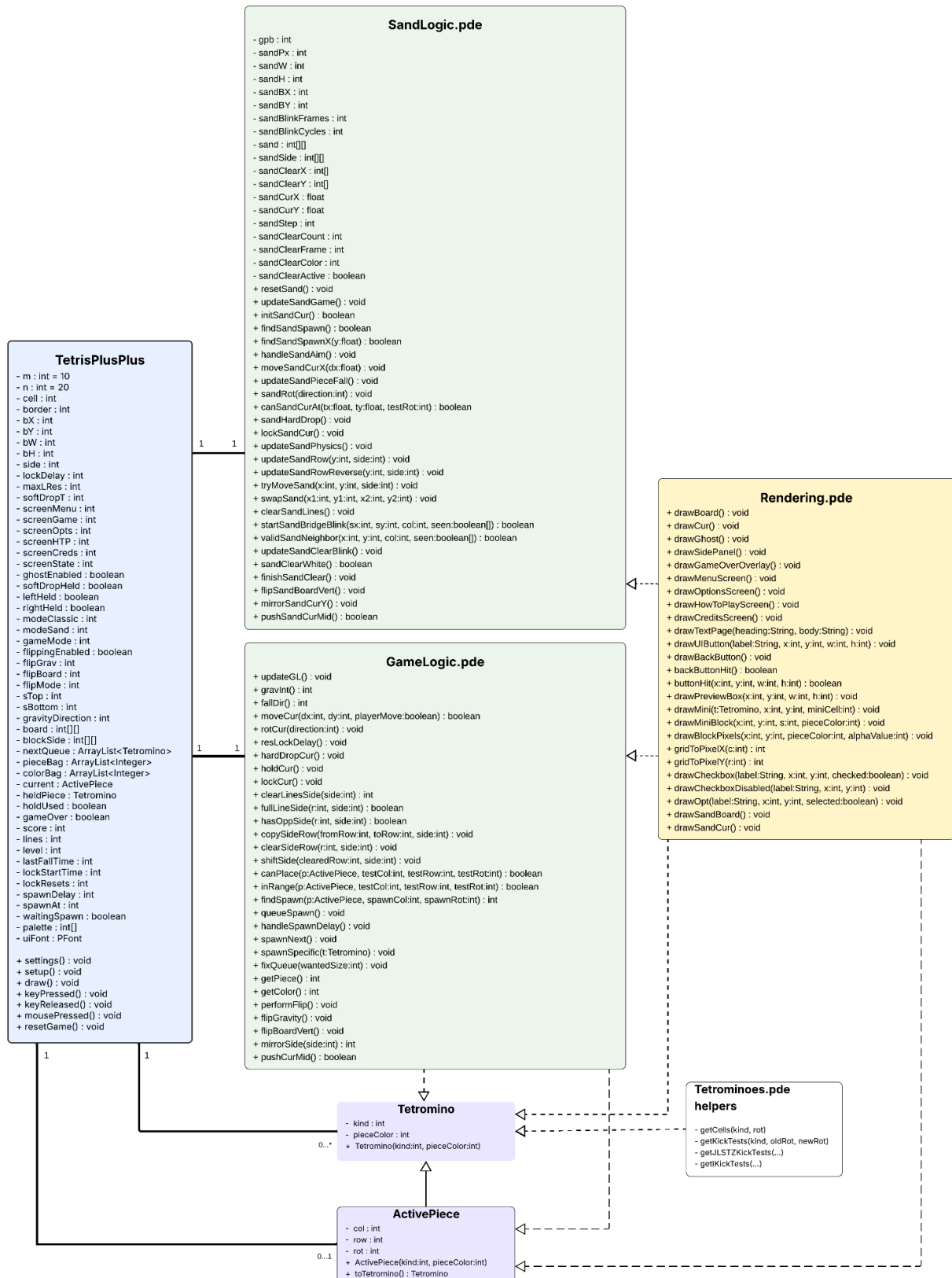
Hudson Dong, Period 6
APCSA 2025-2026

DESCRIPTION:

Tetris++ is a spin-off of the traditional block stacking game of Tetris! Specifically, I build upon the original game with multiple new modes and moves, including “flipping” and “sandifying.” Whether I play normally or disintegrate pieces into piles, or reverse the board or revert gravity, this twist is sure to bring new life into the game! I retain the most important functionalities of gameplay which includes proper board rendering, 7-bag piece and color randomization, multi-row clearing and point accumulation, piece holding and spawning, soft and hard drops, speed and timing, ghost piecing, and gameplay resets. Additionally, I will be implementing two modes: classic and sandification, along with both board flips and gravity flips as a switch at a key press. Consequently I will also adjust spawning and game-over detection mechanics accordingly.

In terms of libraries, the project of course uses the built-in Processing core library for graphics, input, and the game window. Processing functions such as `size()`, `draw()`, `rect()`, `fill()`, `text()`, `keyPressed()`, `keyReleased()`, and `mousePressed()` are used to draw the board, pieces, buttons, text, and handle user input. The project also uses standard Java features such as arrays, `ArrayList`, and `Collections.shuffle()` to store the board, manage pieces, and randomize the 7-bag system.

UML DIAGRAM (LUCIDCHART):



HOW DOES IT WORK?

To start the project, the user, obviously, should open and run the main .pde file.

Upon startup, the user should see the main menu screen titled Tetris++. From there, the user can click "Play" to begin the game, "Options" to toggle the various built-in settings, "How to Play" to read the instructions, or "Credits" to view the credits page.

Once the game begins, tetromino pieces automatically start falling from the top of the board. The player controls each falling piece and tries to place it so that full horizontal lines are created in Classic mode and full horizontal color lines in Sandify. When a row is completely filled, that row disappears, the player earns points, and the blocks or sand above it shift accordingly downward. The game continues until the blocks stack enough so that a new piece has no more space to spawn in.

The controls are:

- Left Arrow - Move the piece left
- Right Arrow - Move the piece right
- Down Arrow - Soft drop -> speeds up the falling of the current piece
- Up Arrow - Hold or swap into held piece slot
- A - Rotate the piece counterclockwise
- S - Hard drop -> instantly drops and locks the current piece
- D - Rotate the piece clockwise
- R - Restart the game
- F - Perform flip -> either board or gravity
- ESC - Return to the menu

As was noted in the introduction, the game is built around two main ways to play: Classic mode and Sandify mode. Both modes use the same basic tetromino idea, where differently shaped pieces spawn one at a time and the player controls where they land. However, the two modes behave very differently once the pieces reach the board. Classic mode behaves as regular Tetris should, where the piece stays as solid square blocks after it locks, conforming to the traditional grid lattice. Sandify mode changes the game into a falling-sand style version, where the piece breaks apart into a myriad of tiny sand particles after it touches down.

In Classic mode, the board is divided into a normal matrix. Each tetromino is made of four square blocks, and the player moves and rotates the active piece until it lands. Once the piece can no longer move farther in the direction of gravity, it locks into the board. After locking, the game checks whether any horizontal rows are complete. A row is considered complete when every column in that row is filled by blocks that belong to the same clearing side. If the row is cleared, the blocks on that side shift toward the cleared space, the player gains score, and the next piece is prepared after a short delay. This delay helps the game feel smoother and prevents pieces from spawning too quickly after one another, over-stacking and self-killing the current game session.

The Classic mode also includes a hold system. When the player presses the hold key (up arrow), the current piece can be stored in the hold slot. If there is already a held piece, the current piece swaps with it. The hold action can only be used once per falling piece, so the player cannot repeatedly swap the same piece before it locks and gain infinite time. This adds strategy because the player can save useful pieces for later, but must choose carefully when to use them.

The next queue shows upcoming tetrominoes. The game uses a piece 7-bag system, meaning all seven tetromino types are shuffled into a bag and drawn from that bag before the bag refills. This prevents

the game from giving the same piece too many times in a row or having a “piece drought” where none of a certain type spawns, making the randomization method of the piece sequence feel more fair. A similar color system is used so pieces can have different colors while still being selected in a controlled way.

The game also includes a ghost piece option in Classic mode. When enabled, the ghost piece shows where the current tetromino would land if the player had dropped it immediately. This helps the player aim and plan placements more accurately. The ghost piece is disabled in Sandify mode because Sandify does not use the same final grid placement behavior as Classic mode, instead breaking into moving particles, so a normal ghost projection would not accurately represent what will happen after the piece touches the sand.

Sandify mode is a separate gameplay mode that uses a much larger particle board instead of the normal Tetris grid. In this mode, the falling tetromino still begins as a solid piece, but it moves more smoothly from left to right rather than snapping only from grid column to grid column. This lets the player aim the piece more precisely before it lands. Once the solid tetromino touches the floor or touches existing sand particles, it breaks apart into many small grains. These grains keep the color of the original piece and become part of the sand simulation.

After a piece breaks apart in Sandify mode, the individual sand particles fall and slide based on simple physics rules. A grain first tries to move in the direction of gravity. If that space is blocked, it may try to slide diagonally around obstacles. This makes the sand pile up in slopes and uneven shapes instead of staying as rigid blocks. The sand continues to animate even while the next piece is waiting to spawn, which helps the board settle naturally and makes the mode feel more alive.

Sandify clearing works differently from Classic clearing. Instead of clearing a full grid row, Sandify clears only when particles of the same color form a continuous connected line from one side of the board to the other. This means a color must create a connected bridge across the board. When the game detects such a bridge, it flood-fills the connected region of that same color to clear every grain. The whole connected group blinks white a few times to show that it is about to disappear. After the blinking effect finishes, that connected color group is removed, and the remaining sand continues falling normally.

The flood fill system is important because Sandify is not based on perfect rows. Since sand particles can pile up unevenly, the game needs to detect connected color regions instead of checking a simple grid line. The flood fill searches through neighboring same-colored particles and determines which particles belong to the connected group. If the group reaches from the left edge to the right edge, it qualifies as a clearing group. This creates a different strategy from Classic mode because the player is not just trying to fill rows. The player is trying to aim colors so that they connect across the entire board. The flip system adds another layer to both modes. In the options screen, flipping can be enabled, and the player can choose between Flip Gravity and Flip Board. Only one of these flip modes can be active at a time. When flipping is enabled, pressing F performs the selected flip action. This makes the game less predictable and gives the player another way to manipulate the board.

Flip Gravity changes the direction that pieces or particles fall. In Classic mode, this means pieces may fall toward the opposite side of the board. The game keeps track of whether blocks belong to the top side or bottom side so that clearing still works correctly after gravity changes. This side logic prevents blocks from the two gravity directions from being treated as one simple stack. Rows are checked from the correct end of the board, and blocks shift according to the side they belong to.

In Sandify mode, Flip Gravity changes the direction that sand particles fall. Particles marked as belonging to one side follow that side’s gravity behavior. This allows sand to fall upward or downward depending on the current flip state. The side marker system is still used so the game can preserve the

difference between sand created from different gravity states. This keeps the Sandify version consistent with the Classic flip behavior.

Flip Board works differently from Flip Gravity. Instead of changing the direction of gravity, Flip Board vertically reverses the board. In Classic mode, the locked blocks are mirrored from top to bottom, and their side markers are also mirrored so that the split-side clearing logic remains correct. Gravity stays downward after a board flip. If the active piece would overlap something after the board flips, the game tries to push the piece toward the middle until a valid empty space is found. If no valid space exists, then the game can end.

In Sandify mode, Flip Board mirrors the particle board vertically. All sand particles move to their opposite vertical positions, and their side markers are mirrored as well. If an active solid tetromino exists during the flip, its position is also adjusted so that it remains aligned with the flipped board. If its new position is blocked, the game searches for a valid position before continuing. This prevents the piece from unfairly spawning or staying inside occupied sand.

Spawning is handled carefully in both modes. The game does not simply create a new piece instantly anywhere above the board. Instead, it checks whether the piece can fit at the proper spawn edge. In normal gravity, pieces spawn at or just below the top of the playable area. In flipped gravity, pieces spawn at or just above the bottom edge, depending on the direction of gravity. If the first spawn position is blocked, the game searches inward from the spawn edge to find a valid position. The game only ends when no valid spawn position can be found.

A short spawn delay is used after a piece locks or breaks apart. This delay makes the game easier to follow visually and prevents too many updates from happening at once. In Classic mode, the delay happens after a piece locks and rows are checked. In Sandify mode, the delay happens after the solid piece breaks into grains. During this time, the board can still update, especially in Sandify, where particles may continue falling and settling before the next piece appears.

The game over condition is based on spawning. The game does not end just because the board is crowded or because a piece touches something. Instead, it ends when a new piece can no longer spawn in any valid position. This is important because both Classic and Sandify can have unusual board states because of flip actions. The spawn search makes the game more fair by giving the player a chance if there is still a legal space for a new piece.

The options screen controls which features are active. The player can choose between Classic and Sandify mode. The player can also enable or disable flip mode and choose whether the flip action affects gravity or the board. Some options only apply in certain modes. For example, ghost piece is useful in Classic mode but disabled in Sandify mode because Sandify uses moving particles instead of fixed block placements.

Overall, the game combines traditional Tetris mechanics with extra systems that change how the player thinks about placement. Classic mode focuses on grid-based stacking, line clearing, holding pieces, and using the ghost piece to plan exact placements. Sandify mode focuses on aiming, particle physics, color connections, and bridge clearing. The flip options affect both modes and add an extra challenge by changing either the direction of gravity or the orientation of the board. Together, these features create a version of Tetris++ that can be played like a standard falling-block puzzle game or like a more experimental sand-physics puzzle game.

Master Functionalities List	Master Variables List
- settings() : void - setup() : void - draw() : void - keyPressed() : void - keyReleased() : void - mousePressed() : void - resetGame() : void - updateGL() : void - gravInt() : int - fallDir() : int - moveCur(dx:int, dy:int, playerMove:boolean) : boolean - rotCur(direction:int) : void - resLockDelay() : void - hardDropCur() : void - holdCur() : void - lockCur() : void - clearLinesSide(side:int) : int - fullLineSide(r:int, side:int) : boolean - hasOppSide(r:int, side:int) : boolean - copySideRow(fromRow:int, toRow:int, side:int) : void - clearSideRow(r:int, side:int) : void - shiftSide(clearedRow:int, side:int) : void - canPlace(p:ActivePiece, testCol:int, testRow:int, testRot:int) : boolean - inRange(p:ActivePiece, testCol:int, testRow:int, testRot:int) : boolean - findSpawn(p:ActivePiece, spawnCol:int, spawnRot:int) : int - queueSpawn() : void - handleSpawnDelay() : void - spawnNext() : void - spawnSpecific(t:Tetromino) : void - fixQueue(wantedSize:int) : void - getPiece() : int - getColor() : int - performFlip() : void	- m : int - n : int - cell : int - border : int - bX : int - bY : int - bW : int - bH : int - side : int - lockDelay : int - maxLRes : int - softDropT : int - screenMenu : int - screenGame : int - screenOpts : int - screenHTP : int - screenCreds : int - screenSize : int - modeClassic : int - modeSand : int - gameMode : int - ghostEnabled : boolean - softDropHeld : boolean - leftHeld : boolean - rightHeld : boolean - flippingEnabled : boolean - flipGrav : int - flipBoard : int - flipMode : int - sTop : int - sBottom : int - gravityDirection : int - board : int[][] - blockSide : int[][] - nextQueue : ArrayList<Tetromino> - pieceBag : ArrayList<Integer> - colorBag : ArrayList<Integer> - current : ActivePiece

- flipGravity() : void
- flipBoardVert() : void
- mirrorSide(side:int) : int
- pushCurMid() : boolean
- resetSand() : void
- updateSandGame() : void
- initSandCur() : boolean
- findSandSpawn() : boolean
- findSandSpawnX(y:float) : boolean
- handleSandAim() : void
- moveSandCurX(dx:float) : void
- updateSandPieceFall() : void
- sandRot(direction:int) : void
- canSandCurAt(tx:float, ty:float, testRot:int) : boolean
- sandHardDrop() : void
- lockSandCur() : void
- updateSandPhysics() : void
- updateSandRow(y:int, side:int) : void
- updateSandRowReverse(y:int, side:int) : void
- tryMoveSand(x:int, y:int, side:int) : void
- swapSand(x1:int, y1:int, x2:int, y2:int) : void
- clearSandLines() : void
- startSandBridgeBlink(sx:int, sy:int, col:int, seen:boolean[]) : boolean
- validSandNeighbor(x:int, y:int, col:int, seen:boolean[]) : boolean
- updateSandClearBlink() : void
- sandClearWhite() : boolean
- finishSandClear() : void
- flipSandBoardVert() : void
- mirrorSandCurY() : void
- pushSandCurMid() : boolean
- drawBoard() : void
- drawCur() : void
- drawGhost() : void
- drawSidePanel() : void
- drawGameOverOverlay() : void

- heldPiece : Tetromino
- holdUsed : boolean
- gameOver : boolean
- score : long
- lines : int
- level : int
- lastFallTime : int
- lockStartTime : int
- lockResets : int
- spawnDelay : int
- spawnAt : int
- waitingSpawn : boolean
- palette : int[]
- uiFont : PFont
- gpb : int
- sandPx : int
- sandW : int
- sandH : int
- sandBX : int
- sandBY : int
- sandBlinkFrames : int
- sandBlinkCycles : int
- sand : int[][]
- sandSide : int[][]
- sandClearX : int[]
- sandClearY : int[]
- sandCurX : float
- sandCurY : float
- sandStep : int
- sandClearCount : int
- sandClearFrame : int
- sandClearColor : int
- sandClearActive : boolean
- I : int
- J : int
- L : int
- O : int
- S : int
- T : int
- Z : int

- drawMenuScreen() : void
- drawOptionsScreen() : void
- drawHowToPlayScreen() : void
- drawCreditsScreen() : void
- drawTextPage(heading:String,
body:String) : void
- drawUIButton(label:String, x:int, y:int,
w:int, h:int) : void
- drawBackButton() : void
- backButtonHit() : boolean
- buttonHit(x:int, y:int, w:int, h:int) :
boolean
- drawPreviewBox(x:int, y:int, w:int, h:int)
: void
- drawMini(t:Tetromino, x:int, y:int,
miniCell:int) : void
- drawMiniBlock(x:int, y:int, s:int,
pieceColor:int) : void
- drawBlockPixels(x:int, y:int,
pieceColor:int, alphaValue:int) : void
- gridToPixelX(c:int) : int
- gridToPixelY(r:int) : int
- drawCheckbox(label:String, x:int, y:int,
checked:boolean) : void
- drawCheckboxDisabled(label:String, x:int,
y:int) : void
- drawOpt(label:String, x:int, y:int,
selected:boolean) : void
- drawSandBoard() : void
- drawSandCur() : void
- Tetromino(kind:int, pieceColor:int)
- ActivePiece(kind:int, pieceColor:int)
- toTetromino() : Tetromino
- getCells(kind:int, rot:int) : int[][]
- getKickTests(kind:int, oldRot:int,
newRot:int) : int[][]
- getJLSTZKickTests(oldRot:int, newRot:int)
: int[][]
- getIKickTests(oldRot:int, newRot:int) :
int[][]

- kindNames : String[]
- kind : int
- pieceColor : int
- col : int
- row : int
- rot : int